

Modularity Clustering

In the previous module, we looked at how we can use eigenvectors to find hidden communities in the graph. In this section, we will explore another very related criteria for finding communities.

In the previous approach, we need to specify the number of communities, our next method does that automatically. This method is called **Modularity clustering** and has been widely used to analyze all kinds of networks.

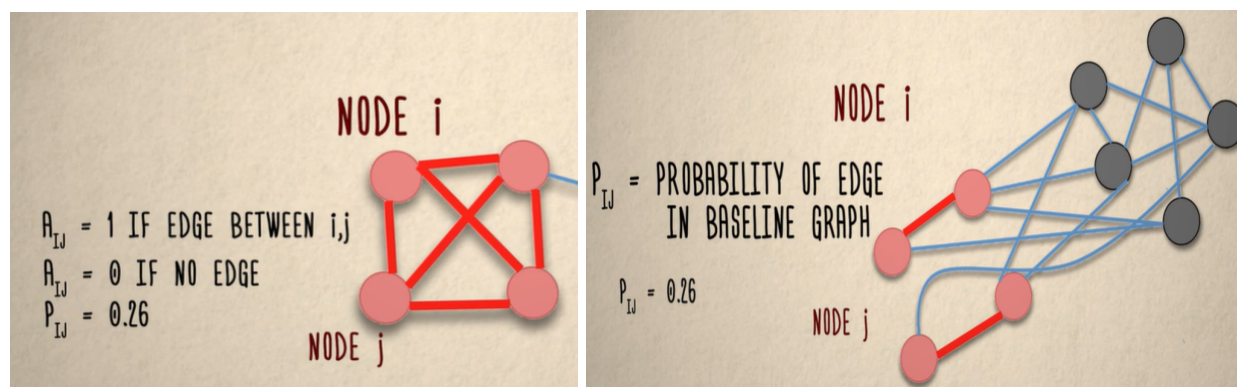
In modularity clustering, we need to find a **modularity score** that we aim to maximize. This score is applied to a given cluster or community. The characteristic idea is to compare clusters or communities to a random baseline graph that shares some properties with our actual graph, such as the number of edges and degree of the nodes.

Good communities should have many more edges than expected by the baseline graph. Let's say we have 2 candidate communities, how to measure their deviation from the baseline.

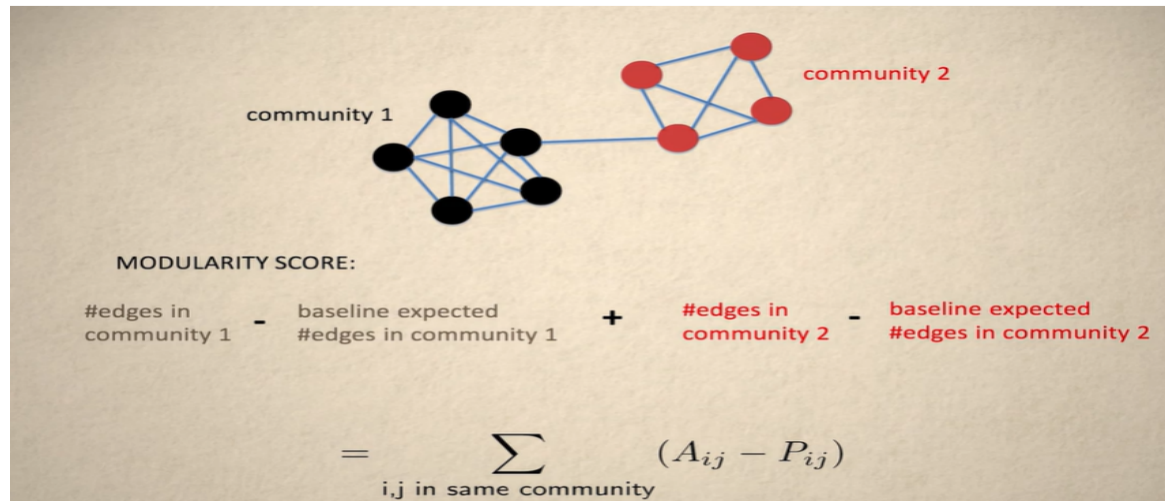
We compute **the number of edges within the community and subtract the expected number of edges based on the baseline model.**

For example: for nodes i and j in the network, we try to find, A_{ij} and P_{ij} , where A_{ij} represents an edge, which is 1 if there is an edge between i and j .

P_{ij} is the expected number of edges between i and j from the baseline graph.



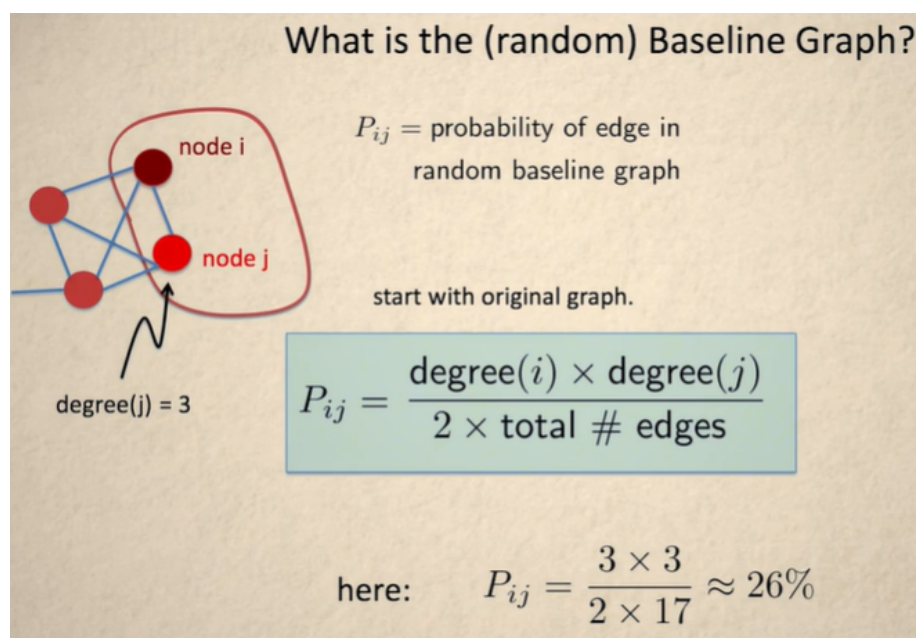
We sum this for all pairs i and j . For each community, we get the actual total number of edges within the community minus the expected number of edges within the community. If this is larger then the community is surprisingly large. This score can even be generalized for an arbitrary number of communities.



Next question is, **what are the expected baseline edges?**

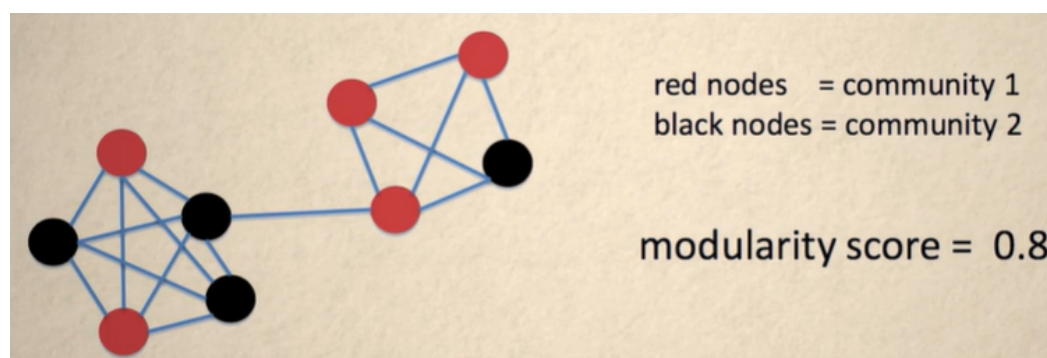
We can calculate the expected number of edges between node i and node j as given in the below image.

From this we get, **that P_{ij} is proportional to the degree of i and degree of j , this means nodes with many edges in the graph are expected to have many edges in the baseline too.**

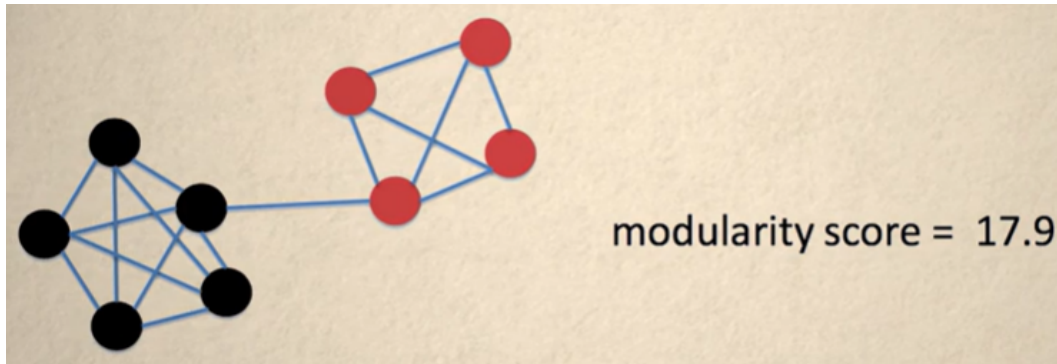


The idea of modularity clustering is that an arbitrary set of nodes say people in social media networks, may have just as many connections as the baselines model, in such cases modularity is close to 0 (Look at the formula for modularity score).

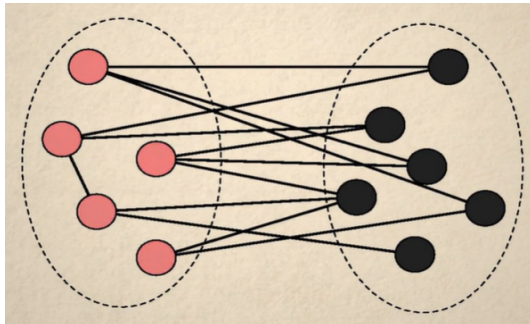
In the example below, we have a low modularity score, divided into meaningless communities.



When we have exceptionally well-separated communities we have a high modularity score.



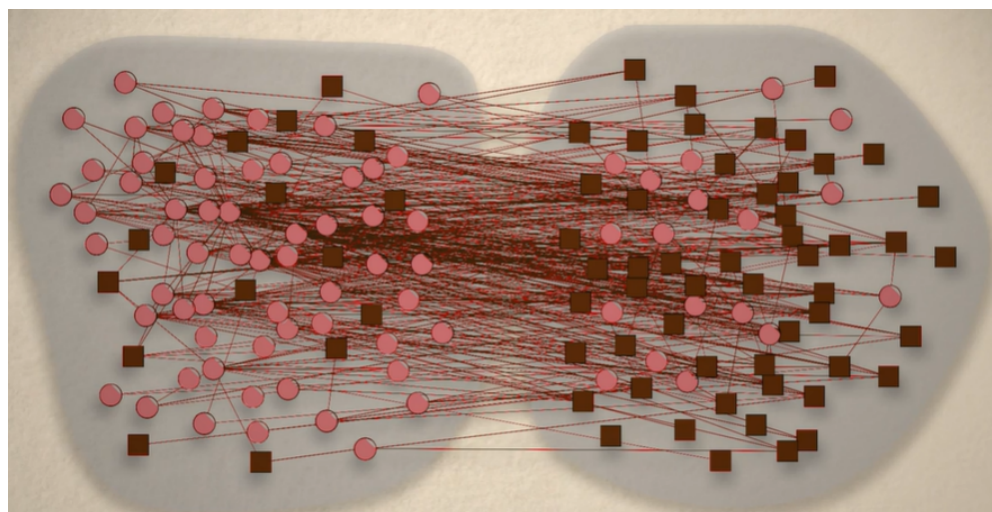
Modularity can be negative too. This happens when the number of nodes in the community is much lower than expected, it may look like the example in the below image.



This may look like the **edge stretch plot with laplacian eigenvectors**, and it is not a coincidence, as this phenomenon is closely related.

Such communities can be useful too, for example in the images given below,
Let's say nodes are a collection of words and they have an edge if they co-occur in a collection of texts. Verbs and nouns typically co-occur in a text, but not 2 nodes and 2 nouns, so this partitioning may separate verbs and nouns.

This means, that if a circle is a noun and a square is a verb, not many circles are connected to circle another circle as 2 nouns usually don't co-occur in a sentence.



Coming back to the dense networks, at the start we mentioned that we don't need to specify the number of clusters for modularity clustering? But how can we then achieve it?

Again, this can be achieved by the magic of eigenvectors.

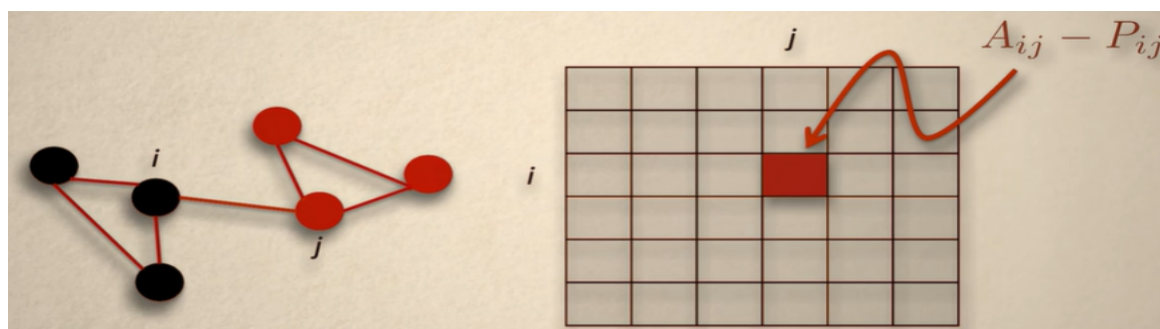
Now, if we want to calculate the eigenvector, again we need a matrix.

This time, this matrix will be similar to the adjacency matrix and Laplacian matrix but slightly different.

Remember, for modularity clustering, we calculate A_{ij} and P_{ij} , where A_{ij} represents an edge, which is 1 if there is an edge between i and j .

P_{ij} is the expected number of edges between i and j from the baseline graph.

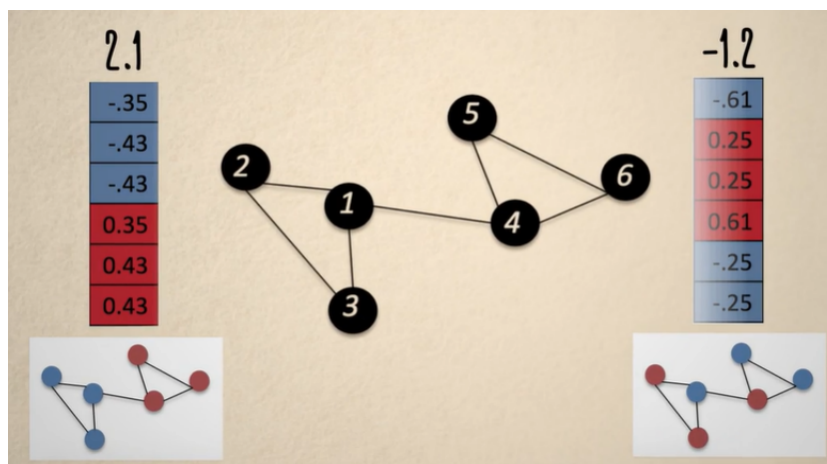
Now, for this matrix, we have an entry for each pair of the nodes, and its value is $A_{ij} - P_{ij}$.



Now, as we have the matrix, we can compute the eigenvectors for this matrix.

Again, each eigenvector assigns a value to each node. For communities, we use the eigenvector with the largest eigenvalue. Then we put all positive nodes in 1 community and all negative nodes in the other. We can even extend this to more eigenvectors and communities. The actual value given by the eigenvector shows how important or central a node in the community is.

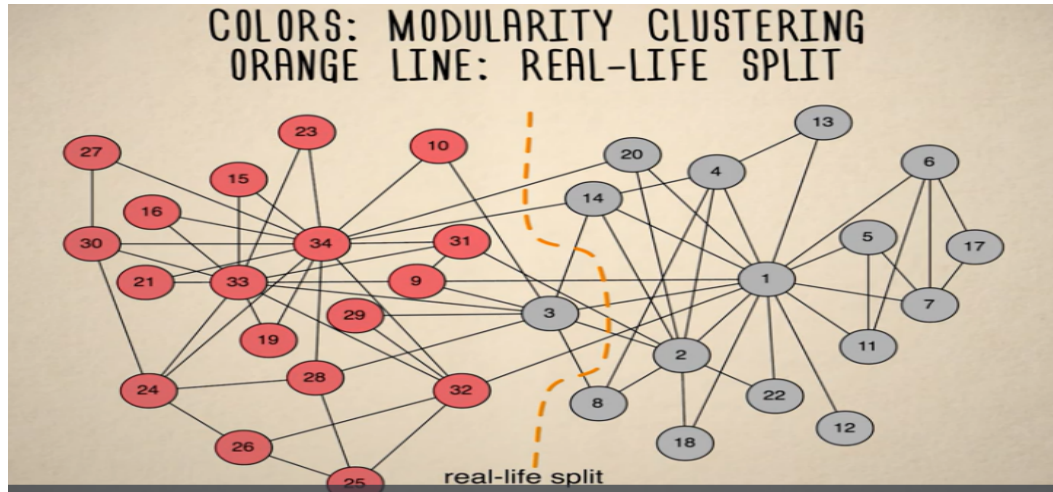
There is another difference in this from our previous clustering method. The eigenvalues here can be negative too. These negative eigenvalues correspond to the low modularity score.



Let's conclude this with a famous example:

Zachary Karate club example: In this example, each node is a member of the karate club at a US university in 1970. The edges indicate the friendship between the members and using modularity clustering, 2 clusters were found. Interestingly this happened after a short period of time, due to some internal dispute the members split apart into 2 groups.

And guess what? The communities found by modularity clustering are exactly the same as they were after the split.



Furthermore, note that if we have more than two communities, then we extend the method to find communities of networks into larger numbers of parts. The standard approach to this problem is repeated division into two: we use the algorithm of the previous section first to divide the network into two parts, then divide those parts, and so forth.